분산 이메일 서비스

Distributed Email Service

메일 한 통을 보내고, 보관하고, 다시 찾기까지



Figure 8.1: Popular email providers

한국어 기술 세미나

설계 범위와 요구사항

사용자 10억 명

메일 송수신과 목록 조회, 읽음·안 읽음 필터를 지원합니다.
제목·발신자·본문 검색, 첨부파일도 포함합니다.

메일을 잃지 않기

일부 서버가 고장 나도 데이터를 보존해야 합니다.
사용자와 메일이 늘어날 때 수평 확장할 수 있어야 합니다.

이번 설계의 경계

사용자 인증은 범위에서 제외합니다.
웹메일은 HTTP로 접근하고, 외부 메일 서버와는 SMTP로 통신합니다.

규모 추정: 저장해야 할 데이터가 먼저 커집니다

약 10만

발신 QPS

10억 명 × 10통 / 일
정확히 계산하면 약 11.6만

730 PB

메타데이터 / 년

10억 × 40통 × 365일 × 50 KB
본문 포함, 첨부파일 제외

1,460 PB

첨부파일 / 년

수신 메일의 20%에 첨부
평균 첨부 크기 500 KB

수신량 40통 / 일과 크기 가정이 저장 용량을 결정합니다.

원문 p.226 · 책의 가정, 복제·색인·백업 비용 제외

메일 프로토콜과 웹메일의 역할

SMTP

메일 서버 사이에서 메시지를 전달합니다.
전통적인 클라이언트의 발신에도 사용합니다.

POP / IMAP

클라이언트가 서버의 메일을 가져오는 프로토콜입니다.
IMAP은 서버에 둔 메일과 여러 기기의 상태를 다루는 데 적합합니다.

HTTPS / MIME

웹메일의 기능은 HTTP API로 제공합니다.
첨부파일을 포함한 메시지 형식은 MIME으로 표현합니다.

수신 서버는 DNS의 MX 레코드로 찾습니다

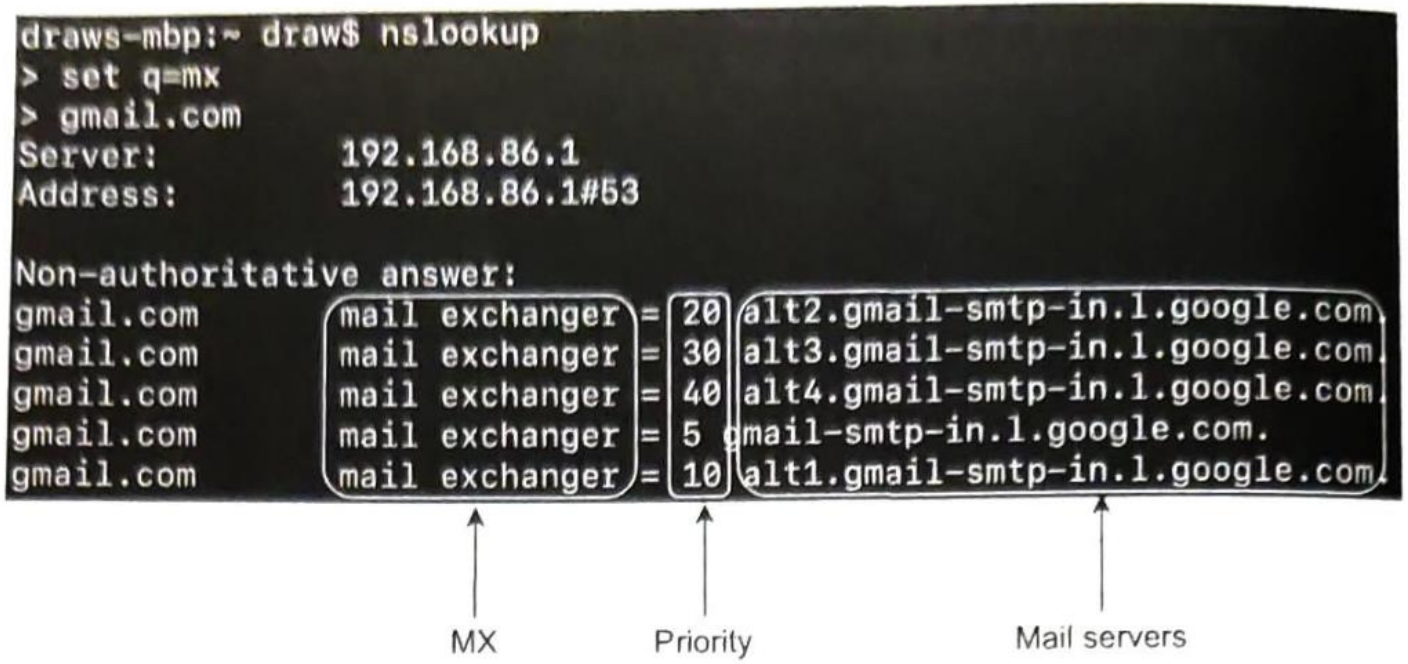


Figure 8.2: MX records

조회 대상

받는 사람 주소의 도메인입니다.

우선순위

숫자가 작은 메일 서버부터 시도합니다.

연결 실패

다음 우선순위 서버를 시도합니다.

원문 p.228 · Figure 8.2

전통적인 메일 서버의 송수신 과정

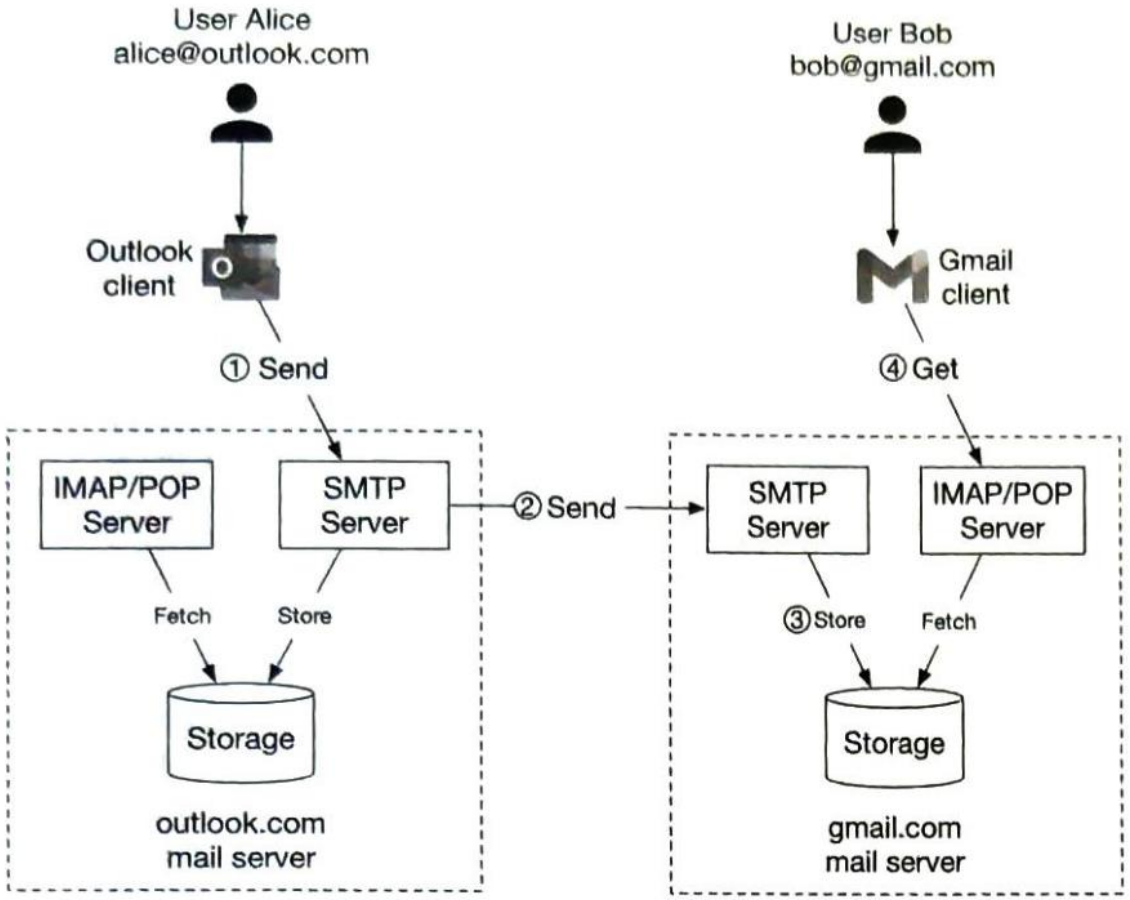


Figure 8.3: Traditional mail servers

보내는 쪽

Alice의 클라이언트가 Outlook 서버로 보냅니다.

서버 간 전달

Outlook 서버가 Gmail 서버에 SMTP로 전달합니다.

받는 쪽

Gmail 서버가 저장하고 Bob의 클라이언트가 조회합니다.

원문 p.229 · Figure 8.3

Maildir와 단일 서버의 한계

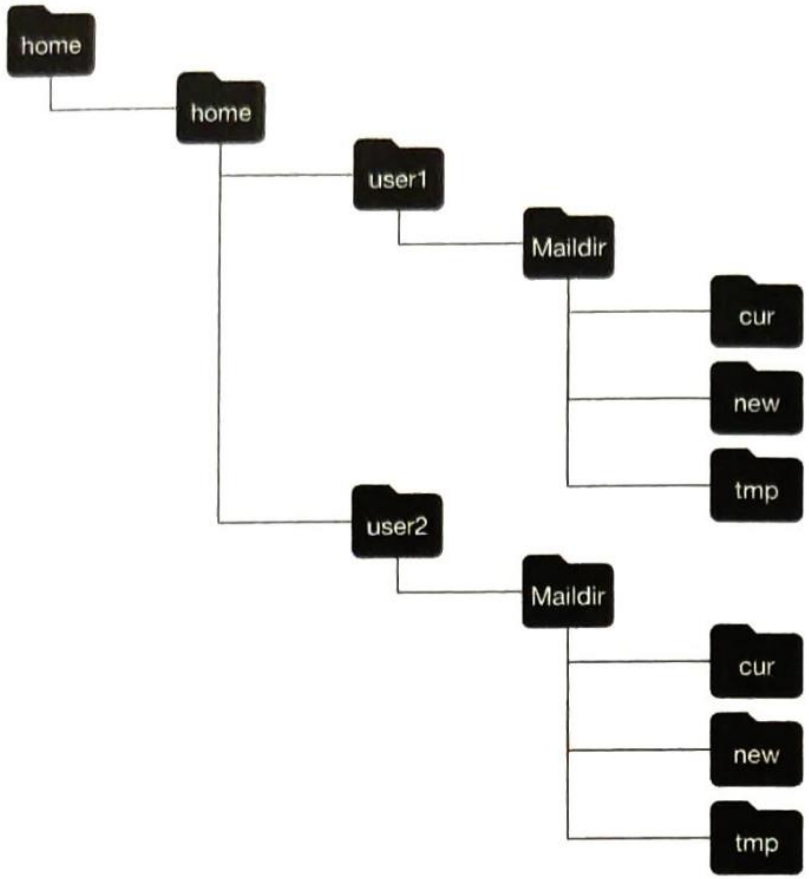


Figure 8.4: Maildir

사용자별 디렉터리

메일을 개별 파일로 저장하는 구조입니다.

파일 수 증가

조회와 백업에서 디스크 I/O 부담이 커집니다.

장애 대응

로컬 디스크만으로는 복제와 고가용성 요구를 충족하기 어렵습니다.

원문 p.230 · Figure 8.4

웹메일 API

POST /v1/messages	메일 보내기
GET /v1/folders	폴더 목록 조회
GET /v1/folders/{folder_id}/messages	폴더 안의 메일 목록
GET /v1/messages/{message_id}	본문과 첨부 정보 조회

목록 API에는 페이지네이션이 필요합니다.

전체 구조: API와 실시간 알림을 분리합니다

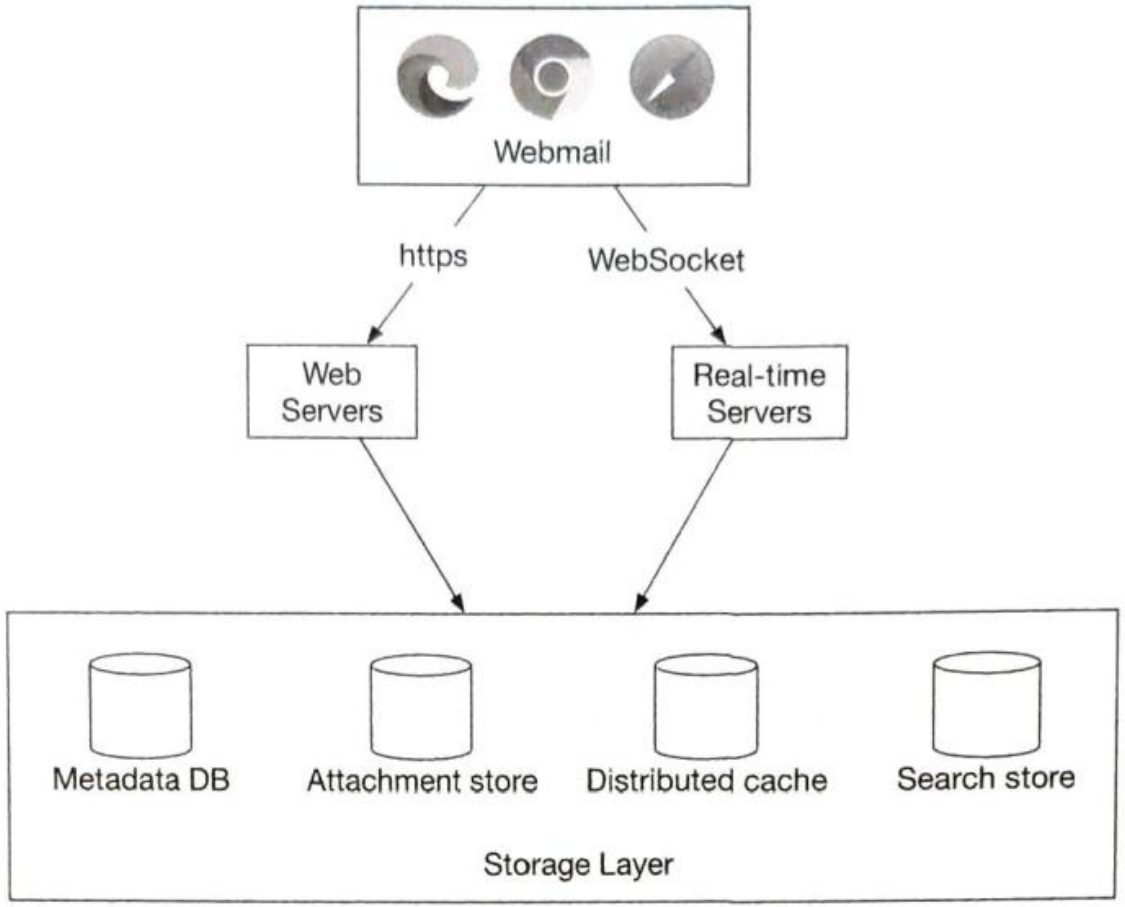


Figure 8.5: High-level design

Web servers

보내기, 폴더 조회, 메일 조회를 처리합니다.

Real-time servers

WebSocket 연결을 유지하고 새 메일을 알립니다.

Storage layer

메타데이터·첨부파일·캐시·검색을 나눠 보관합니다.

저장소를 나누는 기준

메타데이터 DB

제목·본문·발신자·수신자와 메일 상태를 저장합니다.
목록 조회와 사용자별 접근 패턴에 맞춰 설계합니다.

첨부파일 저장소

큰 파일은 S3 같은 오브젝트 스토리지에 둡니다.
큐에는 첨부파일 자체 대신 참조를 넣을 수 있습니다.

캐시와 검색 저장소

자주 읽는 최근 메일은 캐시로 처리합니다.
전문 검색은 역색인을 가진 별도 저장소에서 지원합니다.

메일 발신 흐름

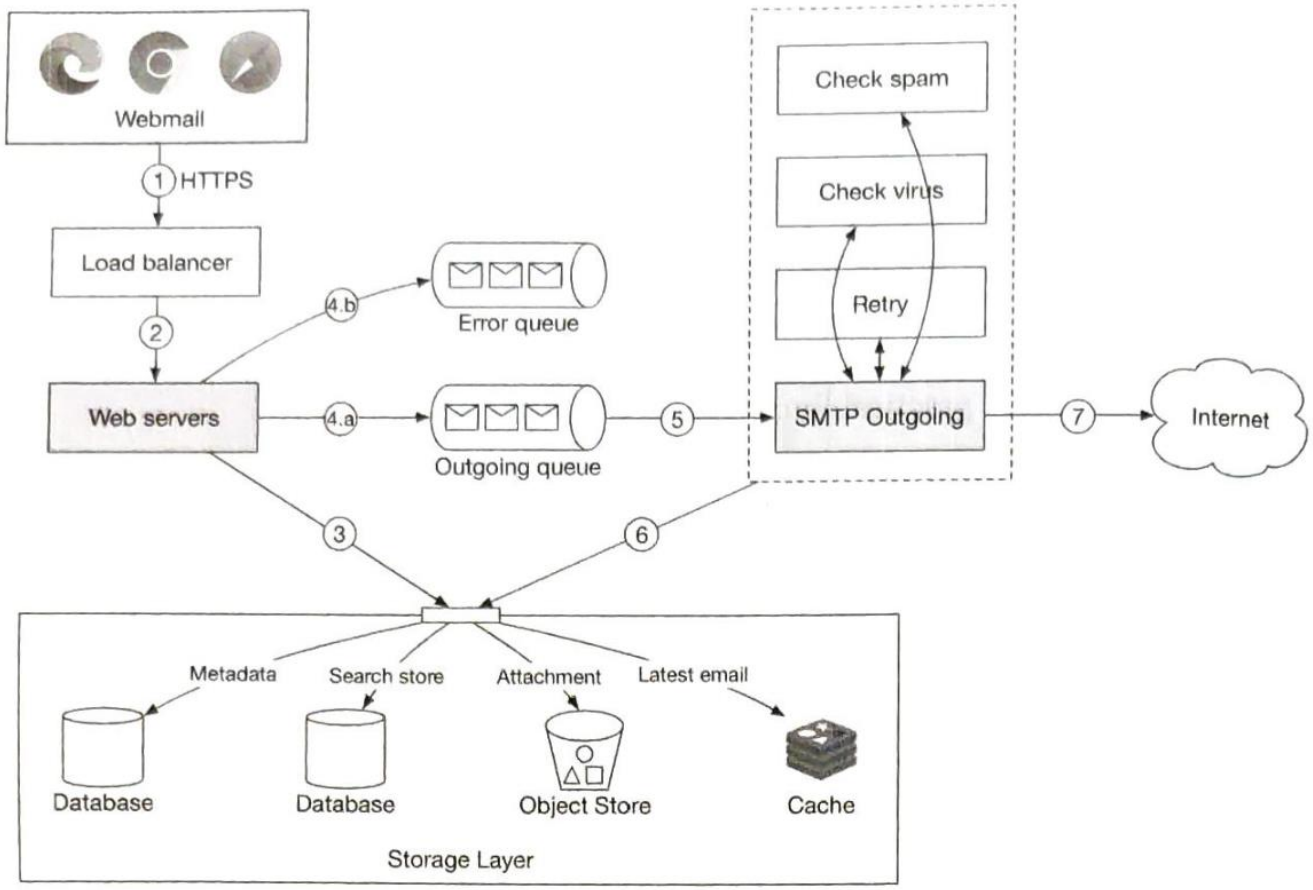


Figure 8.6: Email sending flow

웹 요청은 큐에 넘기고, SMTP 워커가 외부 전달을 맡습니다.

원문 pp.234–235 · Figure 8.6

발신 큐가 길어질 때 확인할 것

상대 서버 장애

수신 서버에 연결하지 못하면 시간을 두고 재시도합니다.
책은 지수 백오프를 제안합니다.

워커 처리량 부족

SMTP 워커를 독립적으로 늘릴 수 있습니다.
웹 서버 수를 늘리는 것과 구분해서 봅니다.

큐의 역할

웹 요청과 외부 메일 전송을 분리합니다.
쌓인 메시지를 관찰해 지연의 원인을 구분해야 합니다.

메일 수신 흐름

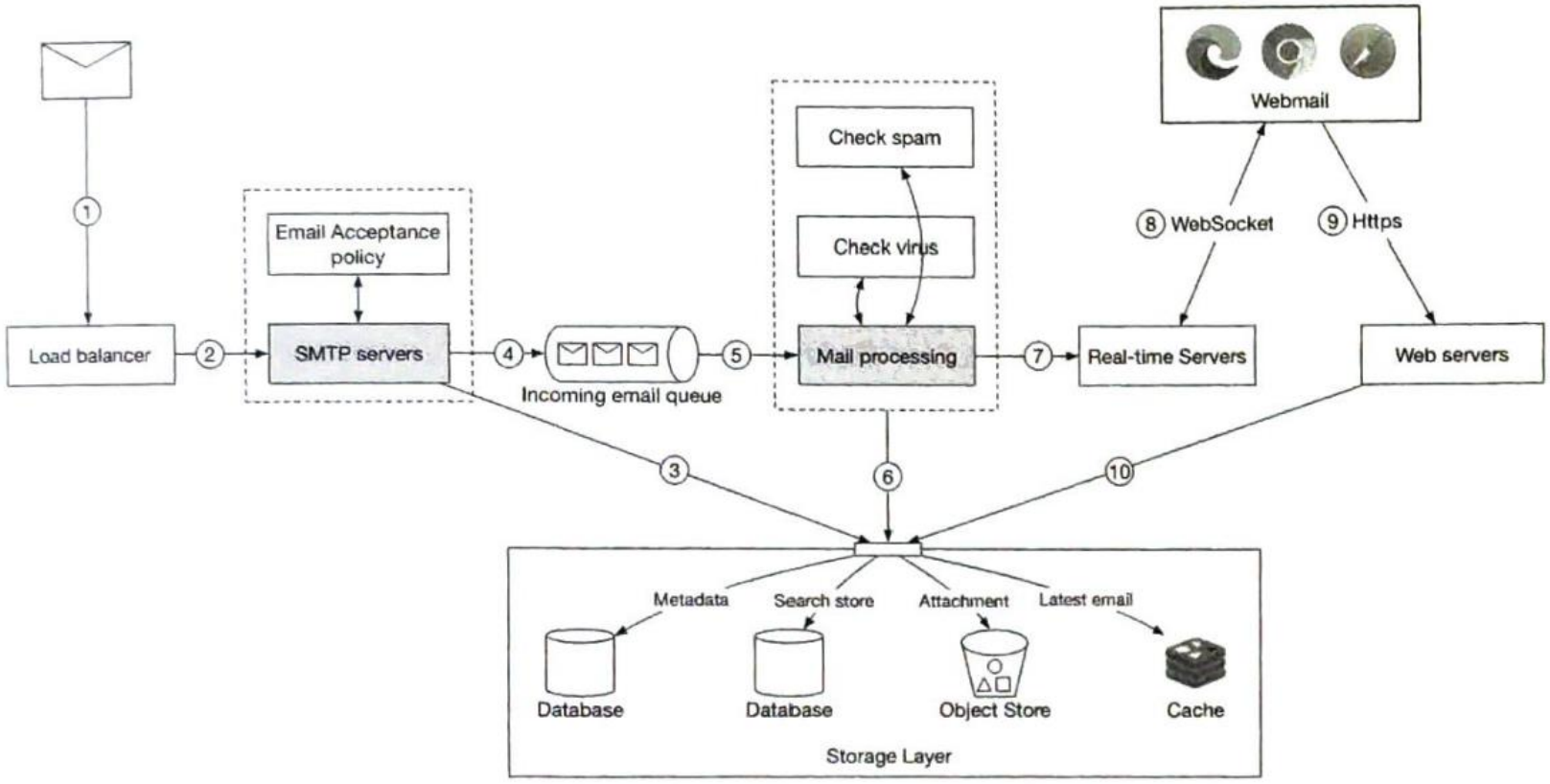


Figure 8.7: Email receiving flow

수신 큐로 처리량을 조절하고, 온라인 여부에 따라 알림·조회를 나눕니다.

원문 pp.235–236 · Figure 8.7

메일 데이터의 접근 패턴

사용자 단위

대부분의 조회와 상태 변경은 자신의 메일함 안에서 일어납니다.
user_id를 분산 기준으로 삼을 수 있습니다.

목록과 본문

작은 헤더는 자주 읽고, 본문은 상대적으로 덜 읽습니다.
목록용 데이터와 상세 데이터를 나눌 이유가 생깁니다.

최근 메일

최근 데이터에 읽기가 몰립니다.
캐시와 저장 계층의 I/O 최적화에 이 특성을 활용합니다.

메타데이터 DB의 선택 기준

큰 본문과 디스크 I/O	단일 컬럼에 수 MB까지 담을 수 있어야 합니다. 메일 접근 패턴에 맞춰 디스크 I/O를 줄여야 합니다.
일관성과 복구	강한 일관성, 장애 내성, 증분 백업을 요구합니다. 제품 이름만으로 이 조건이 충족되는 것은 아닙니다.
책의 선택	NoSQL 형태의 데이터 모델로 설명을 이어갑니다. 초대규모 사업자는 맞춤형 저장소도 고려합니다.

폴더 목록과 메일 목록의 키 설계

folders_by_user		
user_id	UUID	K
folder_id	UUID	
folder_name	TEXT	

Table 8.1: Folders by user

폴더 목록

user_id로 한 사용자의 폴더를 모읍니다.

emails_by_folder		
user_id	UUID	K
folder_id	UUID	K
email_id	TIMEUUID	C↓
from	TEXT	
subject	TEXT	
preview	TEXT	
is_read	BOOLEAN	

Table 8.2: Emails by folder

폴더 안의 최신 메일

(user_id, folder_id) + email_id 내림차순

원문 pp.238–239 · Table 8.1, Table 8.2

메일 상세와 첨부파일 정보

emails_by_user		
user_id	UUID	K
email_id	TIMEUUID	C↓
from	TEXT	
to	LIST<TEXT>	
subject	TEXT	
body	TEXT	
attachments	LIST<filename|size>	

attachments		
email_id	TIMEUUID	C
filename	TEXT	K
url	TEXT	

Table 8.3: Emails by user

상세 데이터

본문과 수신자 목록은 상세 조회에 사용합니다.

첨부파일 참조

파일명·크기·URL 등으로 실제 파일을 연결합니다.

조회 범위

메일 식별자와 함께 소유 사용자 범위도 확인합니다.

원문 p.239 · Table 8.3

읽음·안 읽음 조회를 위한 비정규화

read_emails		
user_id	UUID	K
folder_id	UUID	K
email_id	TIMEUUID	C↓
from	TEXT	
subject	TEXT	
preview	TEXT	

unread_emails		
user_id	UUID	K
folder_id	UUID	K
email_id	TIMEUUID	C↓
from	TEXT	
subject	TEXT	
preview	TEXT	

Table 8.4: Read and unread emails

조회에 맞춘 테이블

read_emails와 unread_emails를 따로 둡니다.

상태 변경

안 읽음 목록에서 지우고 읽음 목록에 넣습니다.

비용의 이동

조회는 단순해지지만 여러 데이터를 맞춰 갱신해야 합니다.

원문 p.240 · Table 8.4

대화 스레드는 헤더의 참조 관계로 묶습니다

```
{
  "headers" {
    "Message-Id": "<7BA04B2A-430C-4D12-8B57-862103C34501@gmail.
        com>",
    "In-Reply-To": "<CAEWTXuPfN=LzECjDJtgY9Vu03kgFvJnJUSHTt6
        TW@gmail.com>",
    "References": ["<7BA04B2A-430C-4D12-8B57-862103C34501@gmail
        .com>"]
  }
}
```

Message-Id	The value of a message ID. It is generated by a client while sending a message.
In-Reply-To	The parent Message-Id to which the message replies.
References	A list of message IDs related to a thread.

Table 8.5: Email header

Message-Id

각 메시지의 식별자입니다.

In-Reply-To

답장 대상인 부모 메시지를 가리킵니다.

References

대화에 관련된 메시지 ID 목록입니다.

원문 p.241 · Table 8.5

메일함 일관성과 장애 전환

메일함의 primary

한 메일함의 변경을 책임지는 주체를 하나로 둡니다.
복제본이 있다고 동시에 쓰기를 허용하는 것은 아닙니다.

전환 중 접근 중단

책은 failover가 끝날 때까지 동기화와 갱신을 멈춥니다.
일시적인 가용성보다 메일함의 일관성을 우선합니다.

설계에서 남는 질문

primary 전환은 어떤 조건에서 확정할까요?
이전 primary가 다시 살아나면 어떻게 다룰까요?

원문 p.241 · 마지막 항목은 세미나 토론 질문

메일이 받은 편지함에 도착하기 위한 운영

발신 IP와 평판

전용 발신 IP를 사용하고 새 IP의 발신량을 서서히 늘립니다.
마케팅 메일과 중요한 메일은 발신 경로를 분리합니다.

스팸 계정 대응

스팸 계정을 빨리 차단해야 발신 서버의 평판을 지킬 수 있습니다.
반송과 신고를 수집하는 피드백 루프가 필요합니다.

발신 도메인 인증

SPF, DKIM, DMARC를 함께 고려합니다.
전송 기능뿐 아니라 상대 사업자의 수신 판단도 중요합니다.

반송과 신고를 처리하는 피드백 루프

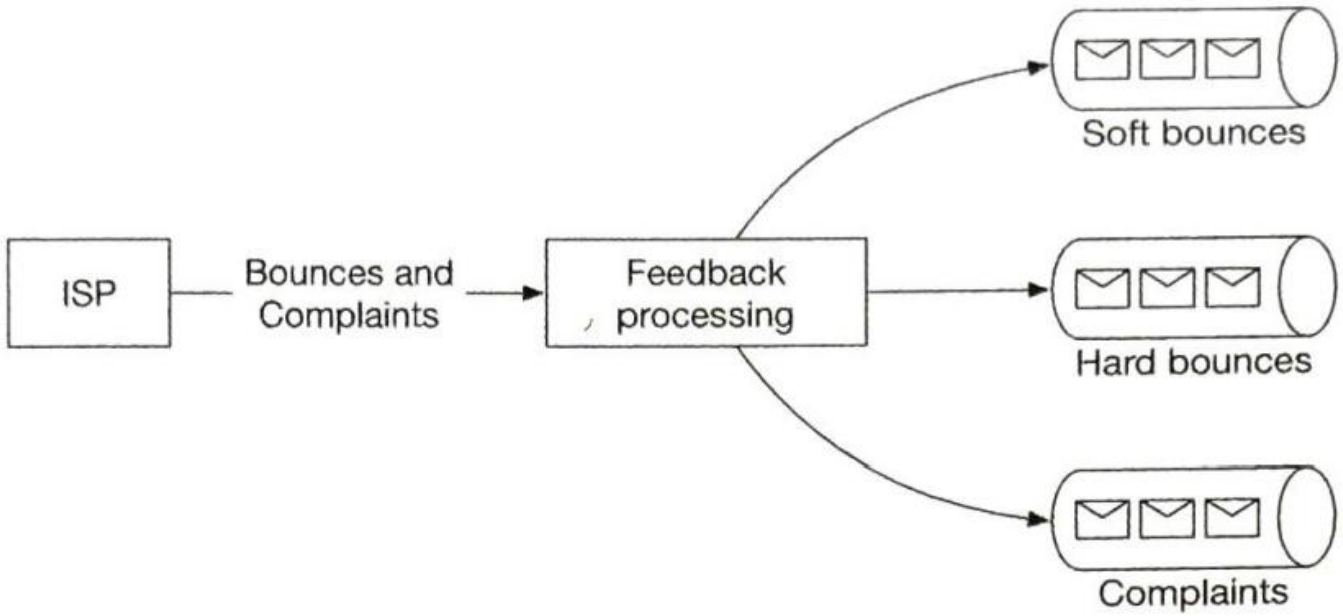


Figure 8.8: Handle feedback loop

Soft bounce

일시적인 사유로 전달하지 못한 경우입니다.

Hard bounce

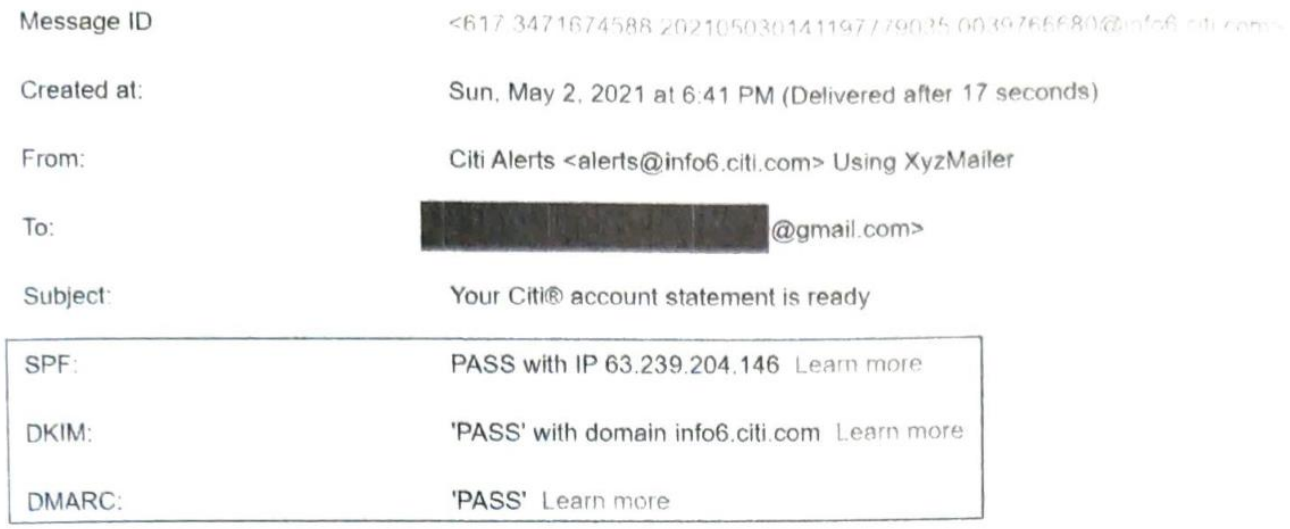
잘못된 수신 주소 등으로 거부된 경우입니다.

Complaint

수신자의 스팸 신고를 수집합니다.

원문 p.242 · Figure 8.8

발신 인증 결과가 메일 헤더에 남습니다



Message ID	<617.3471674588.20210503014119777903.5.0.39766680@info6.citi.com>
Created at:	Sun, May 2, 2021 at 6:41 PM (Delivered after 17 seconds)
From:	Citi Alerts <alerts@info6.citi.com> Using XyzMailer
To:	[redacted]@gmail.com>
Subject:	Your Citi® account statement is ready
SPF:	PASS with IP 63.239.204.146 Learn more
DKIM:	'PASS' with domain info6.citi.com Learn more
DMARC:	'PASS' Learn more

Figure 8.9: An example of a Gmail header

원문 예제

Gmail에서 확인한 인증 결과입니다.

확인할 항목

SPF, DKIM, DMARC의 통과 여부를 보여줍니다.

전달성 관리

발신 인증과 평판, 피드백 처리를 함께 운영합니다.

원문 p.243 · Figure 8.9

메일 검색은 자기 메일함 안에서 정확해야 합니다

	Scope	Sorting	Accuracy
Google search	The whole internet	Sort by relevance	Indexing generally takes time, so some items may not show in the search result immediately.
Email search	User's own email box	Sort by attributes such as time, has attachment, date within, is unread, etc.	Indexing should be near real-time, and the result has to be accurate.

Table 8.6: Google search vs email search

검색 범위

인터넷 전체가 아니라 사용자의 메일함입니다.

정렬과 필터

시간, 첨부 여부, 읽음 상태 같은 속성을 많이 씁니다.

색인 갱신

송신·수신·삭제 때마다 반영해야 하므로 쓰기가 많습니다.

원문 p.243 · Table 8.6

Elasticsearch: 검색 요청과 색인 갱신 경로

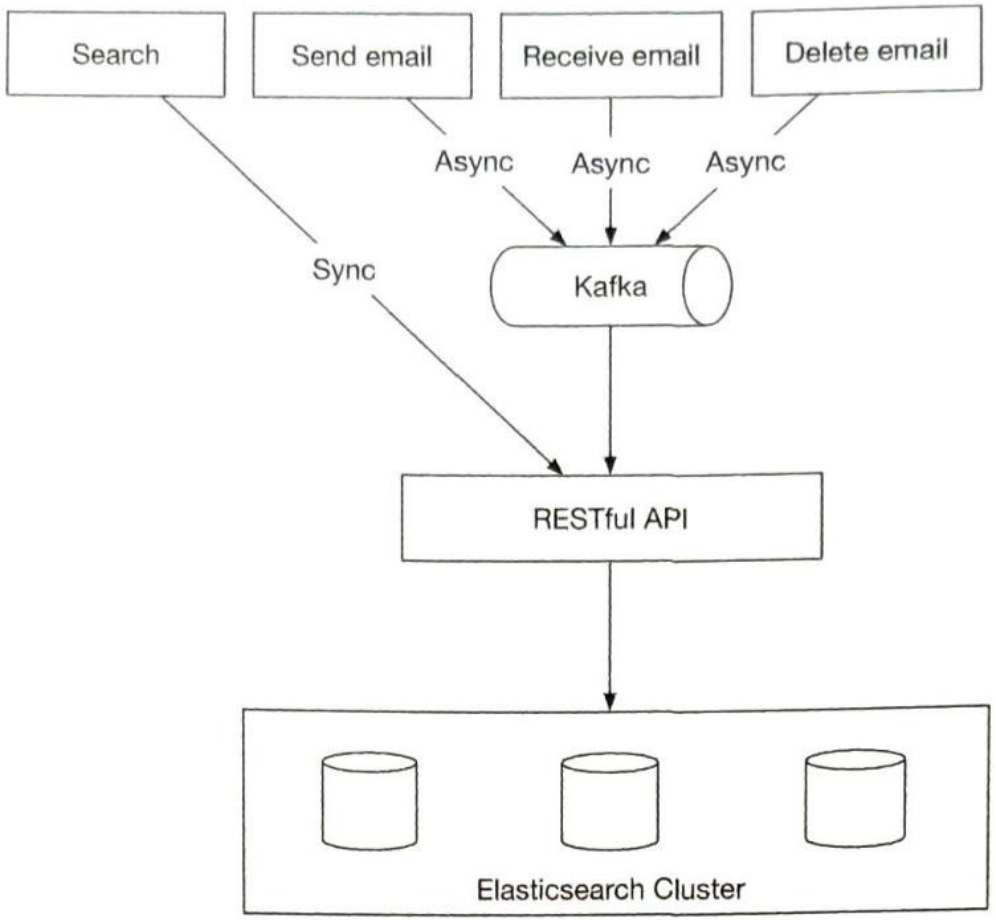


Figure 8.10: Elasticsearch

검색은 동기 요청

사용자가 검색 결과를 기다립니다.

갱신은 비동기 이벤트

송신·수신·삭제 이벤트를 Kafka로 넘깁니다.

운영 과제

원본 메일과 검색 색인의 동기화를 유지해야 합니다.

원문 p.244 · Figure 8.10

맞춤 검색 엔진의 I/O 최적화와 LSM

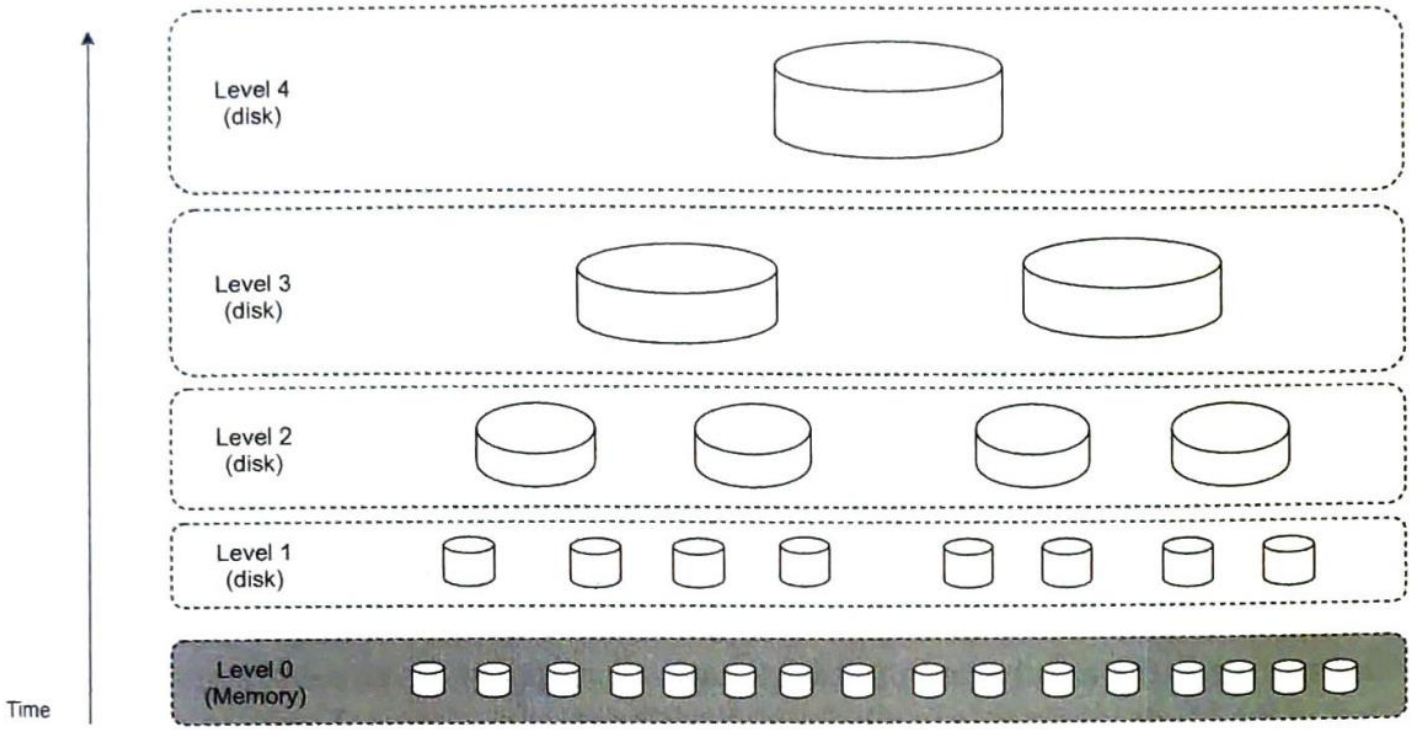


Figure 8.11: LSM tree

메모리에서 시작

새 데이터를 먼저 메모리에 추가합니다.

디스크에 순차 기록

임계점에 도달하면 아래 계층으로 병합합니다.

변경 빈도에 맞춘 분리

거의 바뀌지 않는 본문과 자주 바뀌는 폴더 정보를 나눕니다.

원문 p.245 · Figure 8.11

Elasticsearch와 자체 검색 엔진의 선택

Feature	Elasticsearch	Custom search engine
Scalability	Scalable to some extent	Easier to scale as we can optimize the system for the email use case
System complexity	Need to maintain two different systems: datastore and Elasticsearch	One system
Data consistency	Two copies of data. One in the metadata datastore, and the other in Elasticsearch. Data consistency is hard to maintain	A single copy of data in the metadata datastore
Data loss possible	No. Can rebuild the Elasticsearch index from the primary storage, in case of failure	No
Development effort	Easy to integrate. To support large scale email search, a dedicated Elasticsearch team might be needed	Significant engineering effort is needed to develop a custom email search engine

Table 8.7: Elastic search vs custom search engine

Elasticsearch

통합이 쉽지만 원본과 색인을 따로 운영합니다.

자체 검색 엔진

메일에 맞춘 최적화 여지가 크고 개발 비용도 큽니다.

선택 기준

규모와 운영 인력, 필요한 최적화 수준을 함께 봅니다.

여러 데이터센터로 확장하기

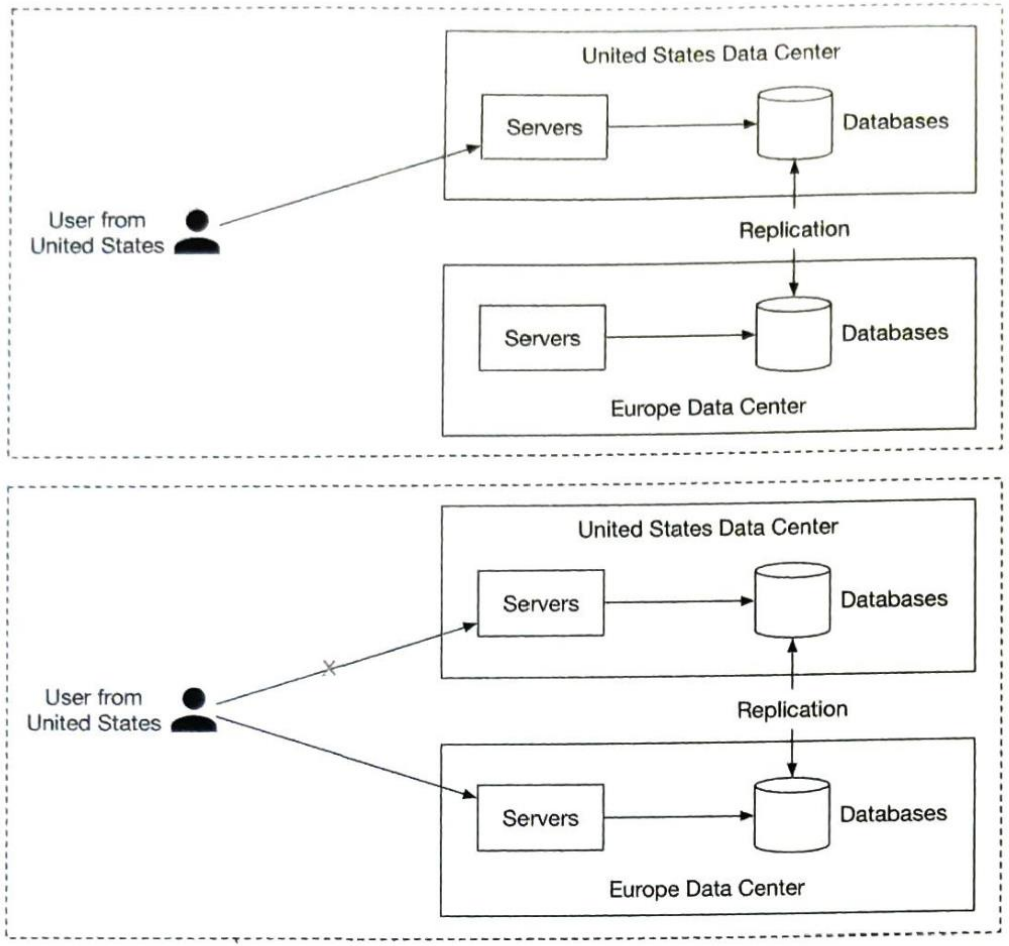


Figure 8.12: Multi-data center setup

평상시

가까운 데이터센터의 메일 서버에 접근합니다.

장애 발생

다른 데이터센터의 복제본으로 접근 경로를 전환합니다.

함께 볼 조건

메일함별 primary와 failover 정책을 유지해야 합니다.

원문 p.247 · Figure 8.12

실제 구현으로 옮길 때 토론할 질문

실패와 재시도	상대 서버가 수신했는데 응답이 끊기면 어떻게 할까요? 재시도로 인한 중복을 어디에서 다룰까요?
상태와 검색	읽음 상태 갱신의 일부만 성공하면 어떻게 복구할까요? 삭제한 메일이 검색 결과에 남아 있으면 어떻게 할까요?
복구와 보관	데이터센터 장애 후 어느 시점까지 복구할 수 있을까요? 첨부파일 중복 제거와 개인정보 보관 정책은 어떻게 맞출까요?

원문 pp.247–248의 확장 주제를 바탕으로 한 토론 질문

오늘 설계에서 기억할 선택

송수신 처리

큐를 두어 외부 전달과 메일 검사를 비동기로 처리합니다.
지연 원인을 구분하고 워커를 독립적으로 확장합니다.

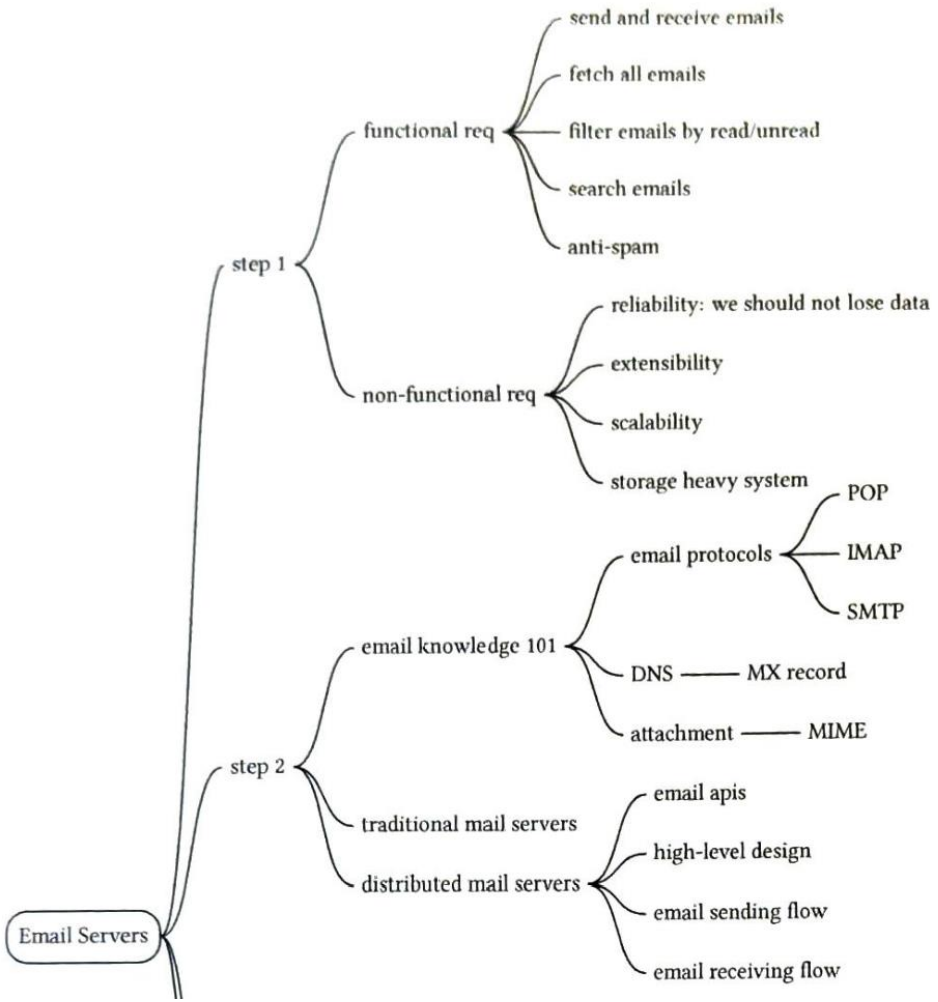
데이터 모델

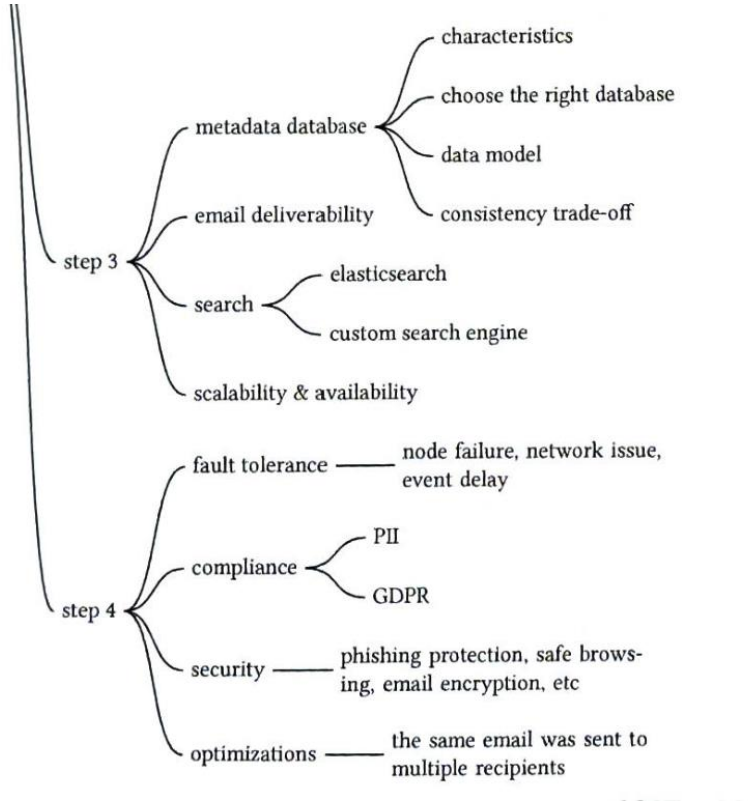
사용자와 조회 패턴을 기준으로 데이터를 모읍니다.
비정규화로 빨라진 조회만큼 갱신 책임도 늘어납니다.

운영과 복구

메일함 일관성, 검색 반영, 발신 평판을 함께 관리합니다.
장애 때 무엇을 멈추고 무엇을 보존할지 정해야 합니다.

부록: 원문 챕터 요약 도식


Email Servers
step 1
functional req
send and receive emails
fetch all emails
filter emails by read/unread
search emails
anti-spam
non-functional req
reliability: we should not lose data
extensibility
scalability
storage heavy system
step 2
email knowledge 101
email protocols
POP
IMAP
SMTP
DNS
MX record
attachment
MIME
traditional mail servers
distributed mail servers
email apis
high-level design
email sending flow
email receiving flow


step 3
metadata database
characteristics
choose the right database
data model
consistency trade-off
email deliverability
search
elasticsearch
custom search engine
scalability & availability
step 4
fault tolerance
node failure, network issue, event delay
compliance
PII
GDPR
security
phishing protection, safe browsing, email encryption, etc
optimizations
the same email was sent to multiple recipients

부록: 원문 그림·표 찾아보기

그림

8.1 서비스 예시 슬라이드 01
8.2 MX 레코드 슬라이드 05
8.3 전통적인 메일 서버 슬라이드 06
8.4 Maildir 슬라이드 07
8.5 전체 구조 슬라이드 09
8.6 / 8.7 발신·수신 슬라이드 11 / 13
8.8 / 8.9 피드백·인증 슬라이드 22 / 23
8.10 / 8.11 검색·LSM 슬라이드 25 / 26
8.12 데이터센터 복제 슬라이드 28

표와 요약 도식

8.1 / 8.2 폴더·메일 목록 슬라이드 16
8.3 메일 상세 슬라이드 17
8.4 읽음·안 읽음 슬라이드 18
8.5 스레드 헤더 슬라이드 19
8.6 검색 특성 비교 슬라이드 24
8.7 검색 구현 비교 슬라이드 27
Chapter Summary 슬라이드 31

책 pp.225–251 = 첨부 PDF pp.228–254